

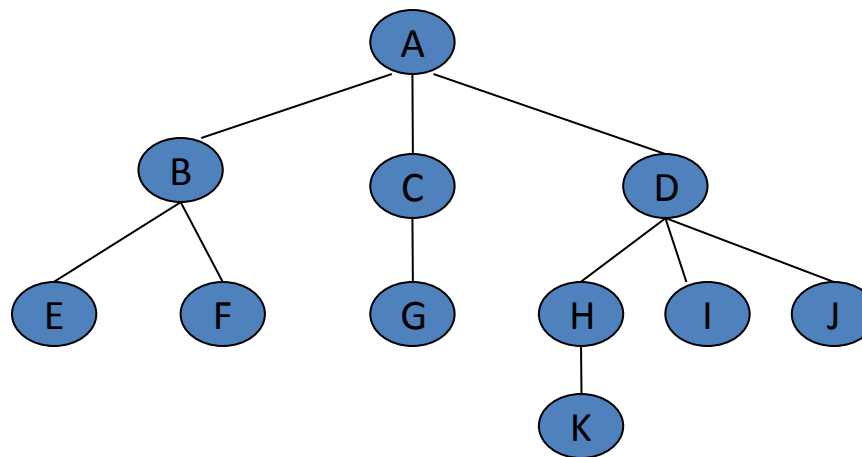
Data Structure

CSE1033

Md. Manowarul Islam
Guest Lecturer
Dept. of CSE, SEU

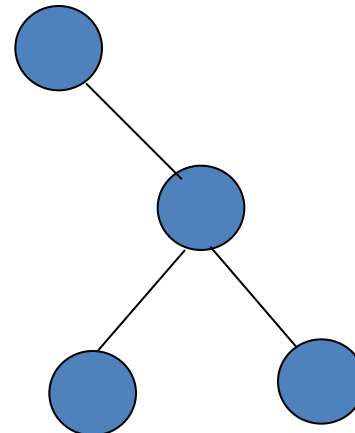
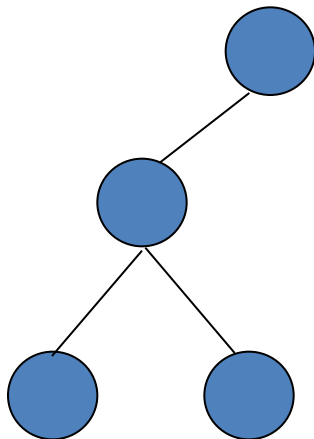
General Tree

- A general tree (a tree) is defined to be a nonempty finite set T of elements, called nodes such that
 - T contains a distinguished element R , called the root of T .
 - The remaining elements of T form an ordered collection of zero or more disjoint trees $T_1, T_2, \dots, T_m$.
- The trees $T_1, T_2, \dots, T_m$ are called subtrees of root R and the roots of $T_1, T_2, \dots, T_m$ are called successors of R .



Difference btn General Tree and Binary Tree

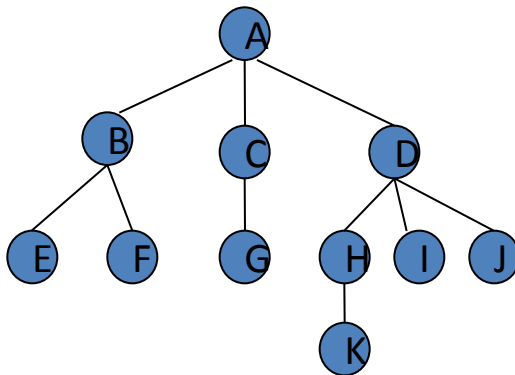
- A binary tree T' is not a special case of a general tree T .
- Suppose a node N has only one child. Then the child is identified as a left child or right child in binary tree T' , but no such distinction exists in a general tree T .



Memory Representation of General Tree

- Suppose T is a general tree. T is maintained in memory by means of a linked representation that uses following three parallel arrays:

- ❑ $\text{INFO}[K] = \text{Information at node } N$
- ❑ $\text{CHILD}[K] = \text{location of the first child of } N.$
- ❑ $\text{SIBL}[K] = \text{location of next sibling of } N$
- ❑ Here K is the location of node N of T.
- ❑ Here ROOT is used as the root of T.



ROOT

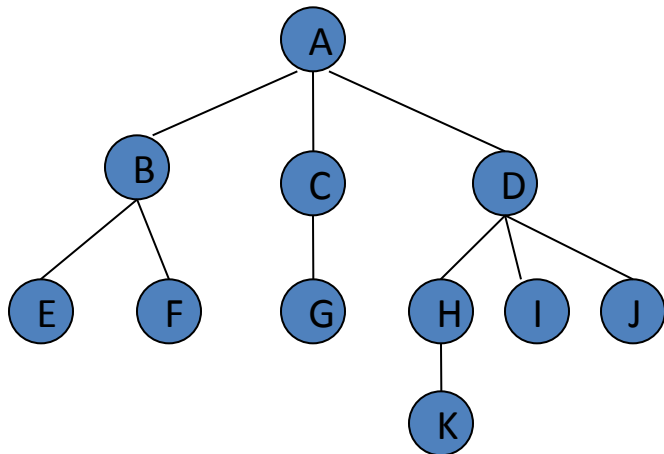
6

	INFO	CHILD	SIBL
1	C	3	13
2	B	5	1
3	G	0	0
4	K	0	0
5	E	0	9
6	A	2	0
7	I	0	12
8			
9	F	0	0
10			
11	H	4	7
12	J	0	0
13	D	11	0

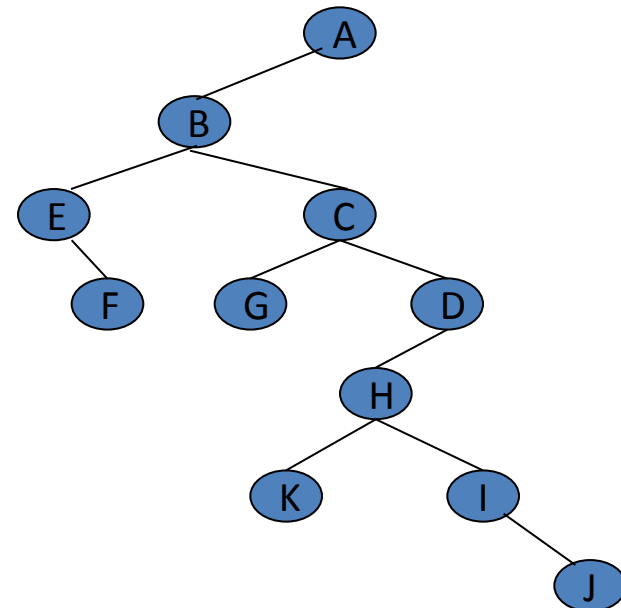
Correspondence between General Tree and Binary Tree

- The root of T' will be the root of T .
- The left child of N' in T' will be the first child of node N in T and the right child of N' in T' will be the next sibling of N in T .

General Tree T

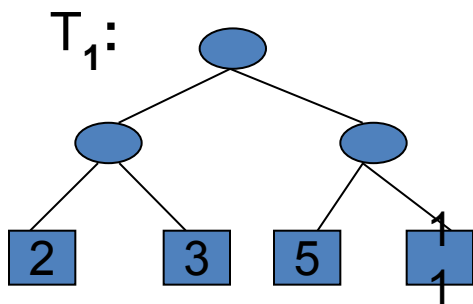


Binary Tree T'

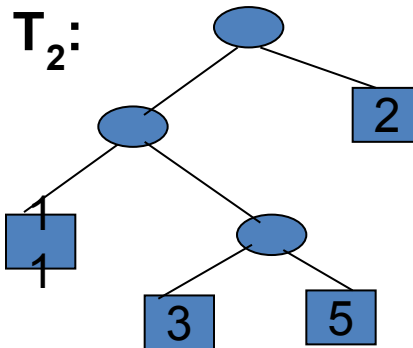


Weighted 2-tree

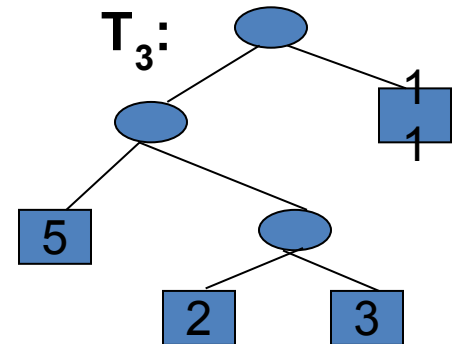
- A weighted 2-tree T is an extended binary tree with n external nodes and each of the external nodes is assigned a nonnegative weight.
- The weighted path length P of 2-tree T is defined to be the sum of all weighted path lengths i.e. $P = W_1L_1 + W_2L_2 + \dots + W_nL_n$.



$$P_1 = 2.2 + 3.2 + 5.2 + 1.1$$
$$= 42$$



$$P_2 = 2.1 + 3.3 + 5.3 + 1.2$$
$$= 48$$



$$P_3 = 2.3 + 3.3 + 5.2 + 1.1$$
$$= 36$$

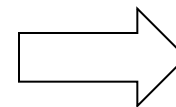
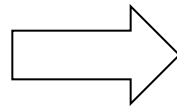
Huffman Codes



Huffman Codes

- For compressing data (sequence of characters)
- Widely used
- Very efficient (saving 20-90%)
- Use a table to keep frequencies of occurrence of characters.
- Output binary string.

“Today’s weather
is nice”



“001 0110 0 0 100
1000 1110”

Encoding with Binary Character Codes

- Encoding

- Concatenate the codewords representing each character in the file

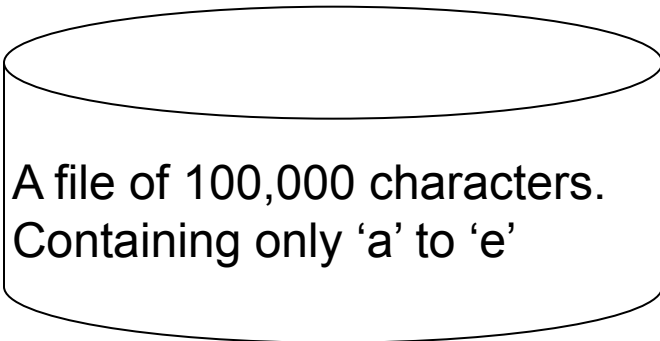
- *E.g.:*

- $a = 0, b = 101, c = 100, d = 111, e = 1101, f = 1100$

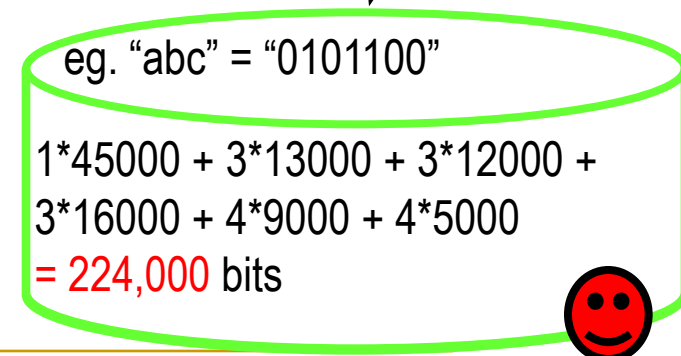
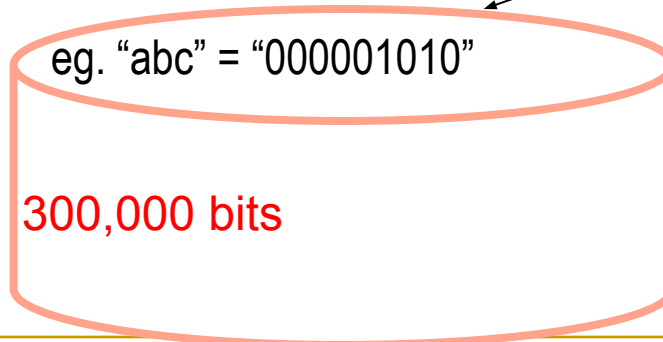
- $abc = 0 \cdot 101 \cdot 100 = 0101100$

Huffman Codes

Example:

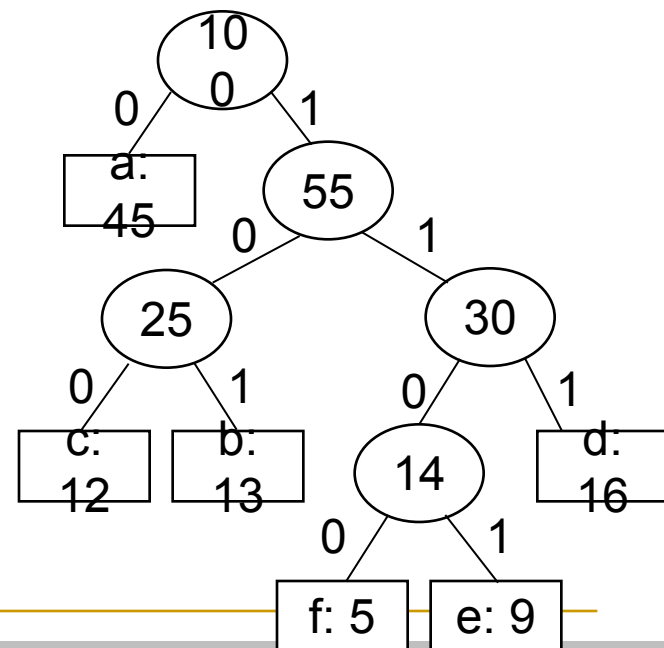
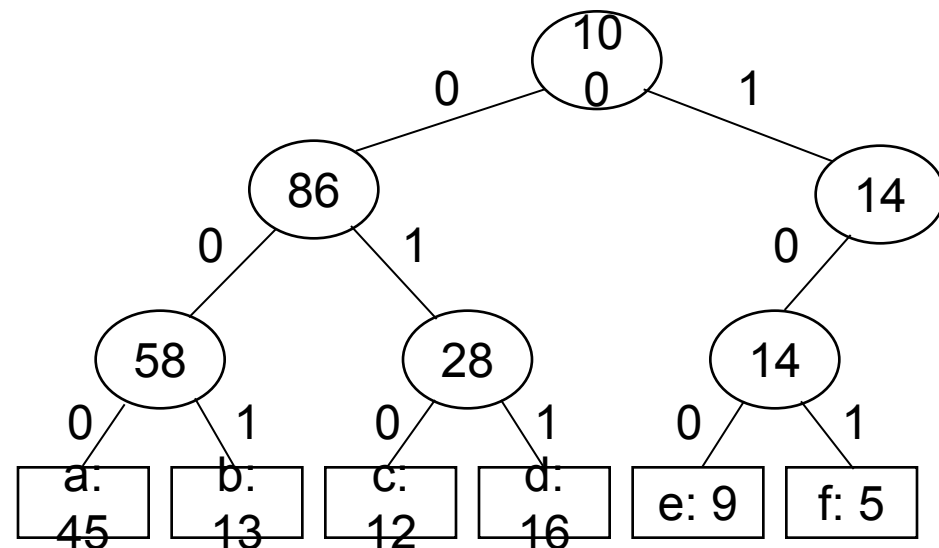


	Frequency	<u>Fixed-length</u> <u>codeword</u>	<u>Variable-length</u> <u>codeword</u>
'a'	45000	000 0	
'b'	13000	001 101	
'c'	12000	010 100	
'd'	16000	011 111	
'e'	9000	100 1101	
'f'	5000	101 1100	



Prefix Code Representation

- Binary tree whose leaves are the given characters
- Binary codeword
 - the path from the root to the character, where 0 means “go to the left child” and 1 means “go to the right child”
- Length of the codeword
 - Length of the path from root to the character leaf (depth of node)

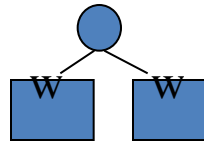


Optimal Codes

- An optimal code is always represented by a **full binary tree**
 - Every non-leaf has two children
 - Fixed-length code is not optimal, variable-length is

Huffman's Algorithm

- Huffman's algorithm is used for building an extended binary tree (2-tree) with a minimum weighted path length from a set of given weights.
 - Suppose w_1 and w_2 are two minimum weights among the n given weights $w_1, w_2, \dots, w_n$.
 - Find a tree T' which gives a solution for the $n-1$ weights $w_1+w_2, w_3, w_4, \dots, w_n$.
 - Then in the tree T' , replace the external nodes w_1+w_2 by the subtree .



- The new 2-tree T is the desired solution

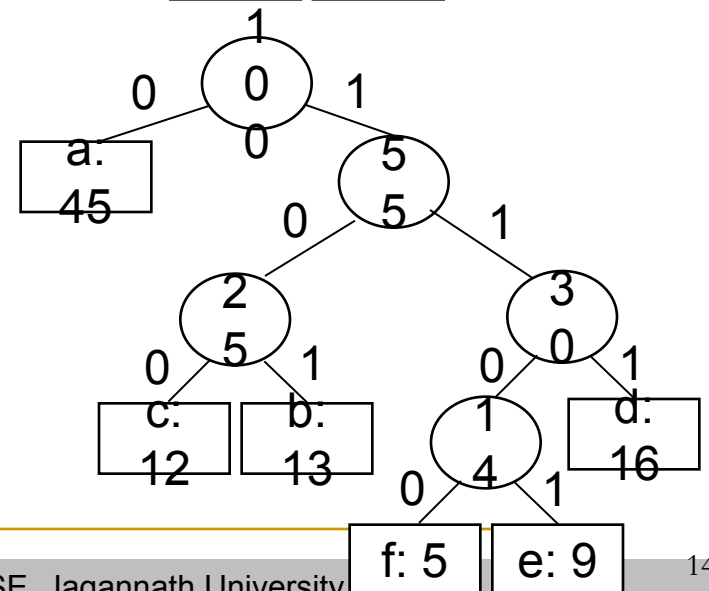
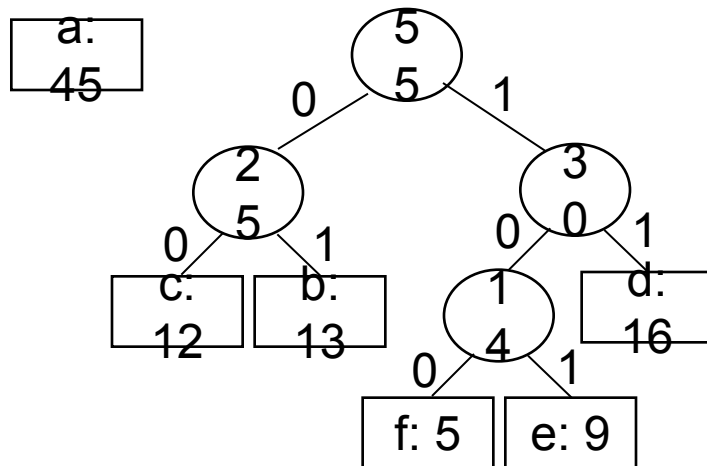
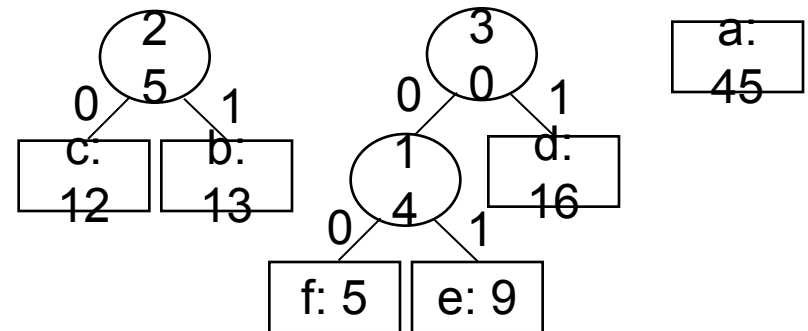
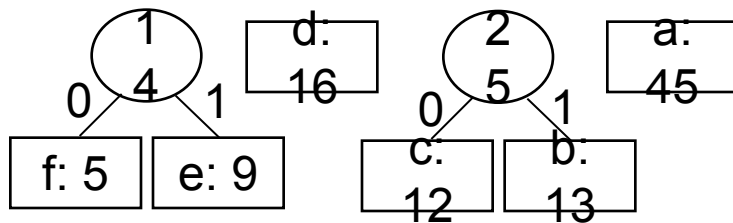
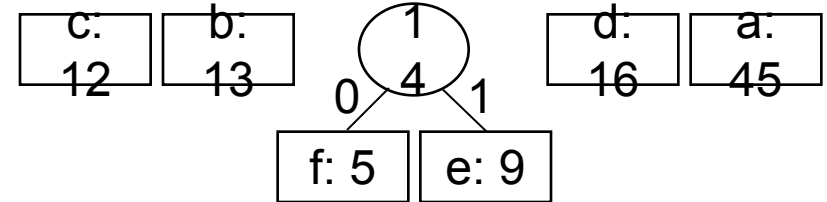
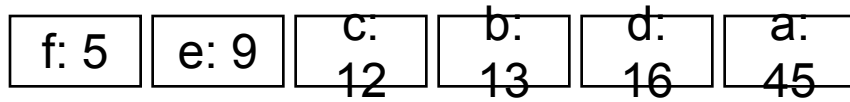
Constructing a Huffman Code

- A greedy algorithm that constructs an optimal prefix code called a **Huffman code**
- Assume that:
 - $\mathcal{C}$ is a set of n characters
 - Each character has a frequency $f(c)$
 - The tree T is built in a bottom up manner
- Idea:

f: 5	e: 9	c: 12	b: 13	d: 16	a: 45
------	------	-------	-------	-------	-------

 - Start with a set of $|\mathcal{C}|$ leaves
 - At each step, merge the two least frequent objects: the frequency of the new node = sum of two frequencies

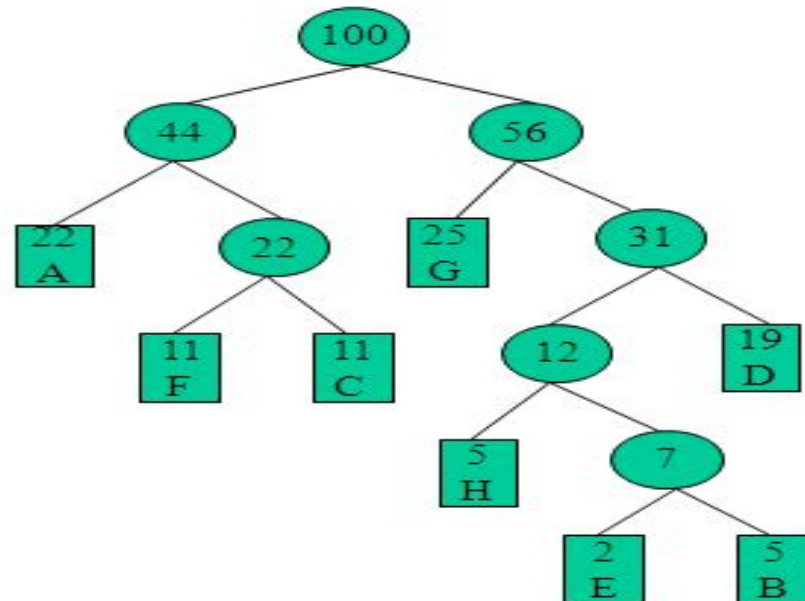
Example



Example

Suppose a set of 8 data items with their weights are given. Construct a 2-tree having minimum weighted path length using Huffman's algorithm.

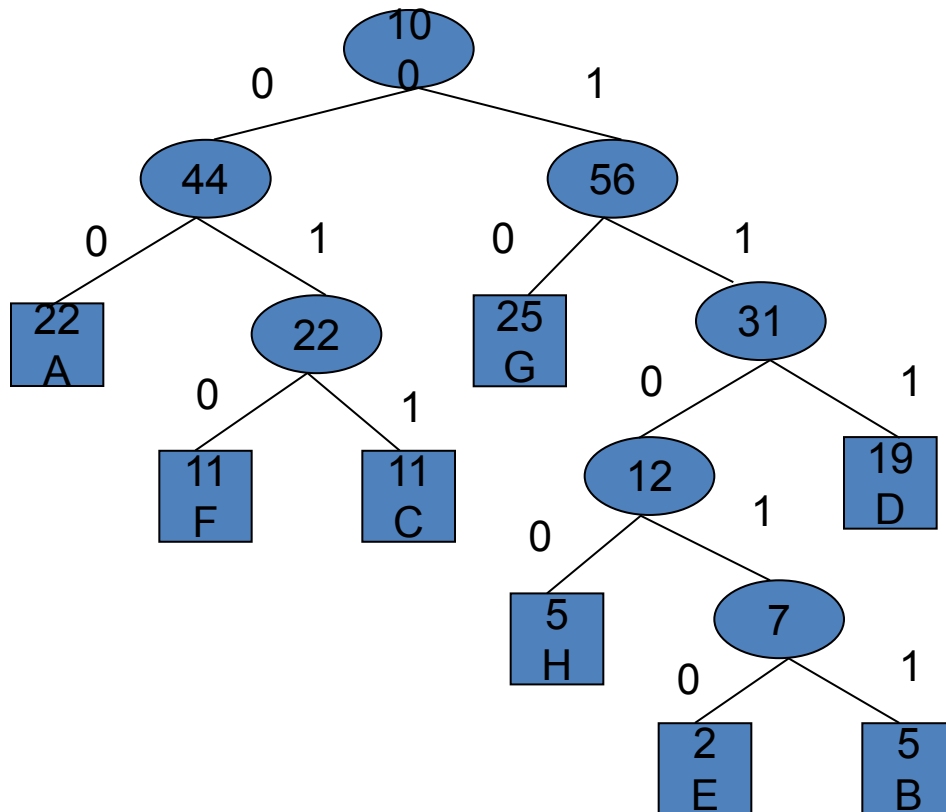
Node	A	B	C	D	E	F	G	H
Weight	22	5	11	19	2	11	25	5



Applications

■ Data Item Coding

- Huffman's algorithm is used to construct a tree that is used for data coding



Code for Data Items

A: 00

B: 11011

C: 011

D: 111

E: 11010

F: 010

G: 10

H: 1100

Applications

■ Data Compression

- Huffman's algorithm is used to construct a tree that is used for data compression.
- The string S “apple pie” is to be encoded.

Char	ASCII	Code
a	97	1100001
p	112	1110000
l	109	1101100
e	101	1100101
i	106	1101001
space	32	1000000

The string “apple pie” can be encoded as follows:

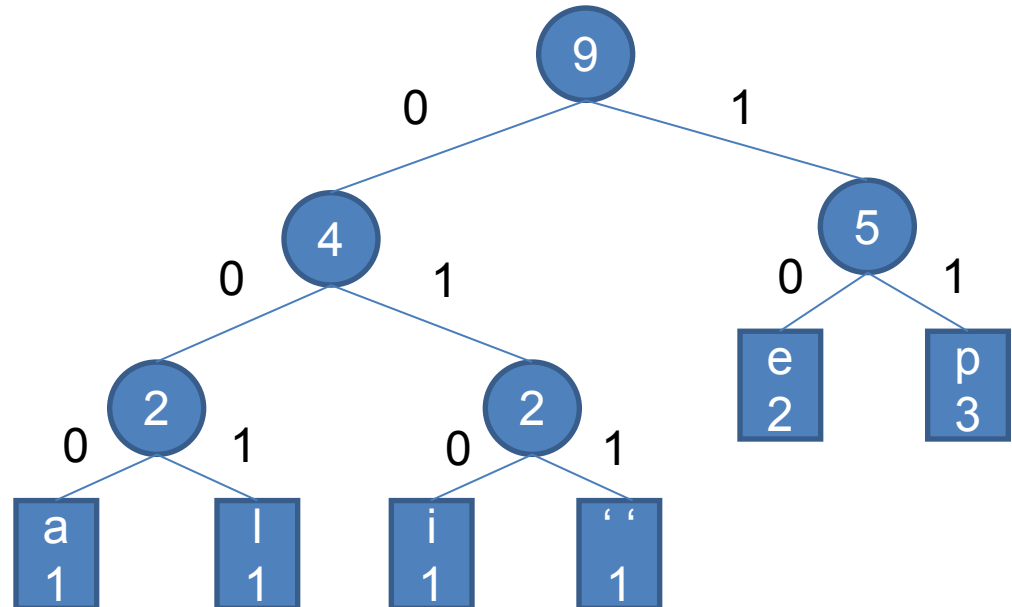
110000111100001110000110110011001011000000111000011010011100101 [64 bits]

Applications

■ Data Compression

- Each character has an associated weight equal to the number of times the character occurs in a file. The characters in S have the following weights:

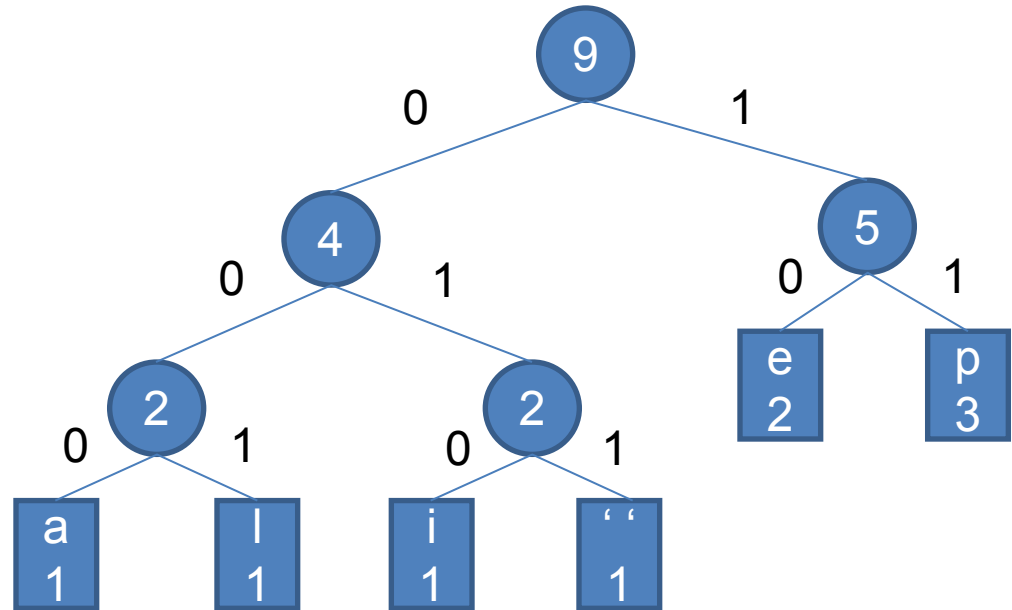
Char	Weights
a	1
p	3
l	1
e	2
i	1
Space ‘ ‘	1



Applications

Char	Weights
a	1
p	3
l	1
e	2
i	1
Space ' '	1

Char	Code
a	000
p	11
l	001
e	10
i	010
Space ' '	011



The string “apple pie” can be encoded as follows:

0001111001100111101010 [21 bits]



End